

PREFACE

General Purpose Hyper-Resilient Edge Supercomputing.

"In the abundance of water, the fool is thirsty." – Robert Nesta Marley, circa 1976

Marabunta started as a charitable High-Performance Computing (HPC) engine meant to give Médecins Sans Frontières (Doctors Without Borders) and other non-governmental organizations the supercomputing power required to predict Ebola outbreaks and manage responses to natural or man-made catastrophes, utilizing donor-provided computational FLOPs and algorithms.

Distributing massive AI and scientific workloads into hostile, infrastructure-poor environments required solving the hardest fundamental problems in distributed systems: bandwidth saturation, state synchronization, and execution trust.

Cambrian Radiation subsequently realized Marabunta's broader applications. Having solved those primitives to ensure the survivability of the network, a generic architecture naturally resulted from those emerging traits.

The system evolved into a substrate for executing any deterministic code, anywhere on Earth, with cryptographic proof of accuracy and physical location. Thus, it seems to have become commercially viable.

MARABUNTA REDONE

TABLE OF CONTENTS

PREFACE: ASYMMETRIC DISTRIBUTED COMPUTE

VOLUME 1: THE THERMODYNAMIC FOUNDATION

- 1.1 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)
- 1.2 The Thermal Guillotine
- 1.3 The PARANOID Micro-Audit

VOLUME 2: THE TOPOLOGY OF CHAOS

- 2.1 Statistical Kademlia Density Estimation
- 2.2 Thermodynamic Gossip Jitter
- 2.3 $O(1)$ Work Resolution (Data Gravity)

VOLUME 3: THE EXECUTION MEMBRANE

- 3.1 Zero-Footprint WASI Streaming
- 3.2 The 64MB Concurrency Cache
- 3.3 Decoupling the Data Plane
- 3.4 Direct S3 Exfiltration

PREFACE: ASYMMETRIC DISTRIBUTED COMPUTE

General Purpose Hyper-Resilient Edge Supercomputing.

The Hyperscaler Paradigm is physically constrained by **Data Gravity** and **Egress Economics**. When processing massive datasets at the point of origin—or when executing logic within restricted, air-gapped jurisdictions—transmitting data to a centralized cluster violates the fundamental laws of network physics and sovereign economics.

Marabunta is an asymmetric execution substrate. It provides a sovereign alternative for environments where the data must remain at the origin.

The Marabunta Swarm is a decentralized mesh designed to coordinate arbitrary heterogeneous topologies. It inhabit high-density enterprise clusters, air-gapped corporate intranets, and consumer silicon alike, harnessing ambient thermodynamic dissipation to execute massive, deterministic workloads.

This document details the mechanical implementation of the core pillars required for sovereign edge compute: * **Physical Identity**: Deriving cryptographic proof of existence from CPU thermodynamic expenditure. * **Deterministic Sandboxing**: Executing untrusted code within strict memory and fuel boundaries. * **Segmented Routing**: Maintaining network consensus across firewalled and high-latency topologies.

This is the technical specification of a production-ready distributed engine.

```
# Illustrative: Monitoring local node integration with the global mesh
$ mrb swarm status --comprehensive
```

VOLUME 1: THE THERMODYNAMIC FOUNDATION

1.1 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)

Centralized Identity Management (IAM) is an architectural bottleneck in decentralized networks. Marabunta replaces the master server with a hardware-anchored identity system.

To participate in the Swarm, a node must bleed computational energy. During the initial boot phase, the node executes the **Chrysalis Grinder**, a memory-hard Proof-of-Work (PoW) algorithm. By saturating the L3 cache and the main memory bus using a unique hardware seed and core identifier, the node derives its sovereign **NodeId**.

This process mathematically neuters the efficiency of custom ASICs, ensuring that identity generation is tied to generic CPU thermodynamic expenditure. A Marabunta identity is a cryptographic receipt of physical work.

```
# Illustrative: Commencing hardware-anchored identity derivation
$ mrb identity derive --difficulty 0x000FFFF ...
```

1.2 The Thermal Guillotine

Because Marabunta can execute untrusted code on consumer-grade hardware, it must assume environments where thermal dissipation is unmanaged. To prevent malicious algorithms or runaway simulations from causing physical damage to host silicon, the system implements the **Thermal Guillotine**.

The orchestrator continuously monitors physical CPU temperatures. If the silicon crosses a critical threshold (e.g., 91°C)—or if sensors are masked by a hypervisor—the daemon executes a hard interruption. By advancing the execution epoch of the underlying WebAssembly engine, the orchestrator terminates the process from outside its own execution boundary.

```
// Illustrative: Hardware health report during a sustained load event
GET /api/v1/node/thermal
{
  "state": "WARNING",
  "temperature": 88.2,
  "throttle_applied": true
}
```

1.3 The PARANOID Micro-Audit

High-priority tasks require consistent memory latency. To filter out weak hardware or virtualized environments that swap memory to disk, the orchestrator enforces a **Micro-Audit** (verify_mode: PARANOID).

The orchestrator demands the node allocate a 256MB matrix and execute a deterministic transformation within 2,500ms. If the node fails this latency test, it is rejected from the high-performance queue. Only bare-metal hardware with high-speed physical RAM is permitted to claim sensitive payloads.

VOLUME 2: THE TOPOLOGY OF CHAOS

2.1 Statistical Kademlia Density Estimation

Marabunta strictly caps its local Kademlia representation to 10,000 peers to preserve host RAM. To accurately monitor a planetary-scale network, the daemon utilizes **Statistical Kademlia Density Estimation**.

By measuring the mathematical XOR distance (prefix length) to its closest neighbors, the node probabilistically extrapolates the global topology. The orchestrator maintains $O(\log N)$ routing efficiency and accurate global telemetry without holding the entire network state in memory.

```
// Illustrative: Fetching extrapolated Swarm topology
GET /api/v1/swarm/density
{
  "known_peers": 10000,
  "estimated_global_scale": 12500400,
  "confidence_interval": 0.95
}
```

2.2 Thermodynamic Gossip Jitter

To prevent global network saturation, Marabunta implements **Adaptive Gossip Jitter**. The background daemon dynamically evaluates the extrapolated node count. As the network scales, the base heartbeat interval aggressively decelerates, and a randomized exponential backoff is injected. This preserves ISP backbones while maintaining eventual consistency.

2.3 $O(1)$ Work Resolution (Data Gravity)

The `KnowledgeStore` maintains a strict $O(1)$ indexed queue of unassigned chunks. When a node requests work, the CPU instantly pops contiguous task fragments off the stack in constant time, unlocking 100% of the silicon for the execution sandbox.

VOLUME 3: THE EXECUTION MEMBRANE

3.1 Zero-Footprint WASI Streaming

WebAssembly sandboxes operate under a 4GB linear memory boundary. Marabunta bypasses this limit using **Zero-Footprint WASI Streaming**.

The physics algorithm invokes a custom host-function, passing a 64-bit offset. The Rust orchestrator intercepts the call and executes a synchronous HTTP Range request against the remote datalake, injecting the targeted byte slice directly into the WASM memory buffer. Massive datasets are processed sequentially without exhausting the host's physical storage.

```
// Illustrative: Logic for the Marabunta streaming host-function
fn mrb_dataset_stream_read(ptr: *mut u8, len: u32, offset: u64) → u32;
```

3.2 The 64MB Concurrency Cache

To achieve native throughput, the `WorkEngine` initializes a single, shared 64MB asynchronous cache buffer. Concurrent sandboxes executing against the same datalake mechanically share this single window, eliminating 98% of redundant network handshakes and preserving host RAM.

3.3 Decoupling the Data Plane

Marabunta strictly decouples the Control Plane from the Data Plane. When a large payload completes, the `WorkEngine` writes the payload locally to disk and gossips only a 32-byte hash pointer across the Kademlia DHT.

3.4 Direct S3 Exfiltration

To prevent master-node disk implosion, Marabunta utilizes **Direct S3 Exfiltration**. Job manifests can distribute presigned cloud upload URLs. Worker nodes bypass the orchestrator entirely, transmitting physics results directly back to the cloud storage bucket.

```
# Illustrative: Verifying decoupled result manifest hashes
$ mrb job results --id ab9f2... --verify-hashes
```